

- PART 1**

۱. خب نتیجه بدست آوردیم :

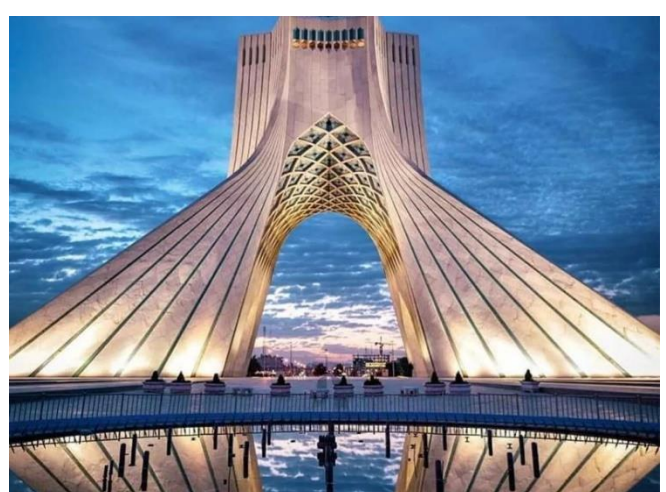
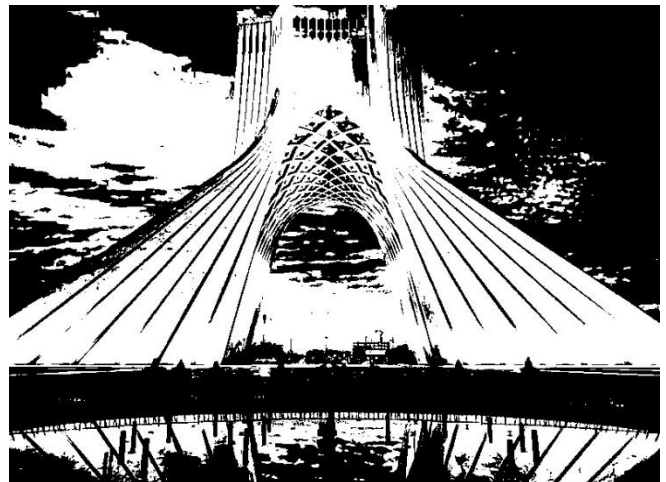
```
(633, 800, 3)
uint8
<class 'numpy.ndarray'>
```

که همانطور مشخص است عکس آپلود شده ۳ کانال دارد.

۲. تصویری که خوانده شده به شکل BGR خوانده شده است و همانطور هم نمایش داده میشود و وقتی آن را به RGB عوض میکنیم رنگ آن معکوس می شود

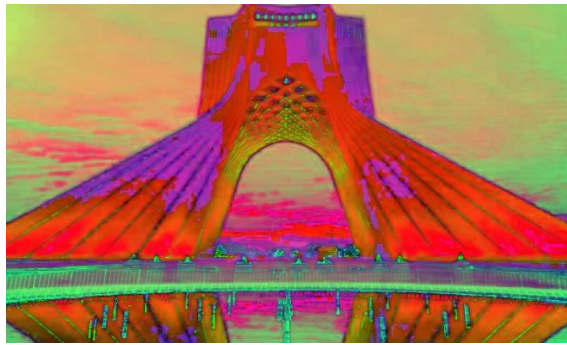
در تصویر خاکستری هر سه کانال به یک کانال تبدیل شده اند (یک ماتریس) و این باعث میشه که میزان شدت روشنایی بیان کنه

در تصویر باینری عکس خاکستری را گرفته و مقدار های بین ۱۲۷ تا ۲۵۵ را سفید کامل و بقیه را سیاه کامل می کنیم



تصاویر هستند به ترتیب : باینری – خاکستری – اورجینال(خوانده شده) – آر جی بی(معکوس شدن کانال ها

۳. در HSV تصویر را به جای اینکه مانند RGB بر اساس سه رنگ تفکیک کنیم میاییم بر اساس فاز (H) شدت و خلوص رنگ (S) و میزان روشنایی و تیرگی رنگ آن را تفکیک میکنیم

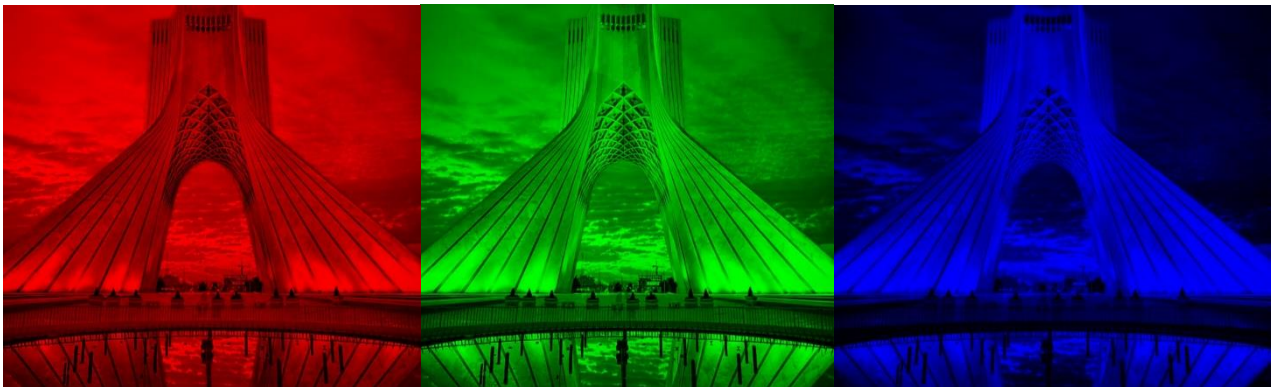


تصویر HSV

۴. در این بخش من یک مشکلی داشتم و اینکه به سو مولفه جدا دقیقاً منظور چیست چون وقتی سه مولفه جدا می‌کنید تصویر سیاه سفید میشه و شاید خواسته سوال رنگی بودنش باشه علت خاکستری شدنش اینه ما سه ماتریس داریم جدا می‌کنیم و اگر هر ماتریس به دستور imshow بدهیم نمایش دهد اعداد را به شکل سیاه سفید نمایش می‌دهد ولی اگر دو ماتریس تمام صفر اضافه کنیم رنگی برداشت می‌کند. اینجا سعی شده هر دو کار انجام شود.



از راست به چپ به ترتیب مولفه اول دوم سوم تصویر RGB



مولفه های تصویر RGB (به صورت رنگی)

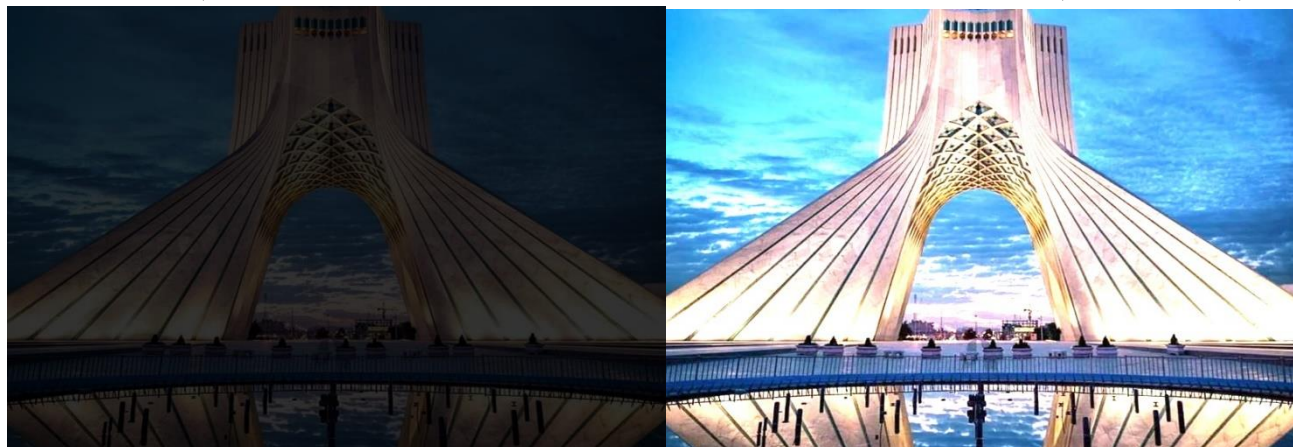


به ترتیب از راست به چپ مولفه H مولفه S مولفه V

۵. کنتراست معیاری برای میزان تفاوت بین روشن ترین بخش یک تصویر با تیره ترین بخش یک تصویر است به عبارتی هر چه کنتراست بیشتر باشد میزان تفاوت تاریکی و روشنی بخش های تصویر افزایش می یابد

برای این کار میتوان از انحراف معیار میان پیکسل ها استفاده کرد که دستورش هم در پایتون موجود است و برای افزایش و کاهش کنتراست باید کاری کنیم اعداد پیکسل ها بدون هیچ تغییر میانگینی از حول میانگین دورتر شوند یعنی باید انحراف معیار را افزایش داد برای این کار می توان صرفا اعداد را در ضربی ضرب کرد اینطور اعداد بزرگ رشد بیشتری خواهند داشت نسبت به اعداد کوچک تر ولی باید حواسمان باشد این ضریب به هر مقدار نمی تواند باشد زیرا اگر از یک مقداری بیش از اندازه شود همه اعداد به اندازه کافی بزرگ شده و از آنجا که پیکسل نهایت تا عدد ۲۵۵ را می تواند داشته باشد بخش بزرگی از اعداد ماتریس تصویر ۲۵۵ خواهند شد و با افزایش بیش از اندازه ضریب کنتراست یا احراف معیار افزایش می یابد البته راه های دیگری هم هست برای این کار ولی این اولین مسیری بود که به ذهنم گذشت

این هم لازمه اضافه کنم برای انحراف معیار حساب کردن تصویر از تصویر خاکستری شده استفاده می کنیم

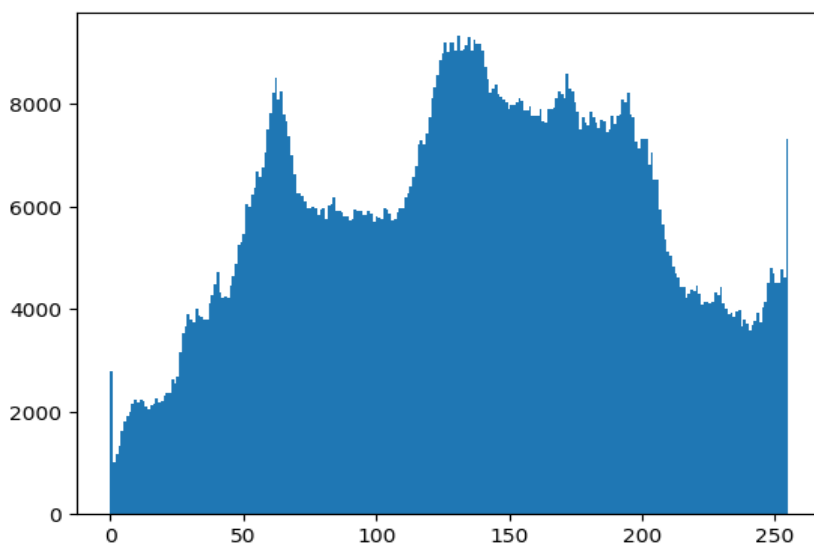


تصاویر با کنتراست های مختلف

Original Contrast: 54.84205130125853
Higher Contrast: 62.31837678713477
Lower Contrast: 10.989012253508205

اعداد کنتراست تصویر ها

۶. نمودار هیستوگرام به ما نشان میدهد اگر مثلا تصویر به سمت تاریکی باشد بیشتر آن در سمت چپ یا اگر روشن باشد بیشتر آن در سمت راست است به عبارتی محور افقی برای میزان روشنایی-تیرگی است (اعداد ۰-۲۵۵) و محور افقی میزان تکرار است



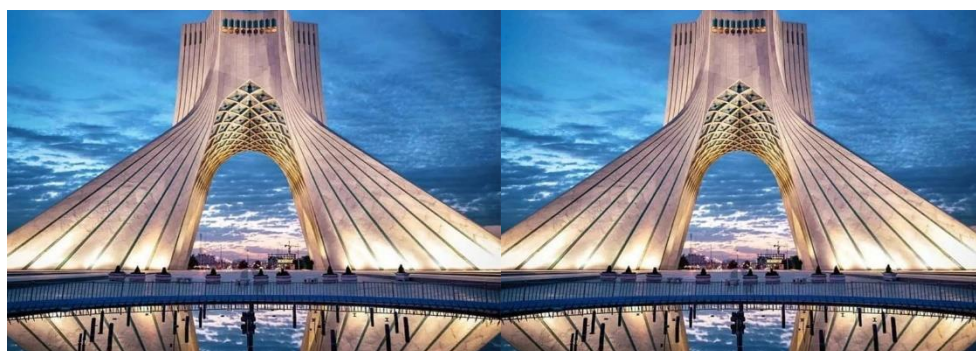
هیستوگرام یک تصویر

۷. روش Histogram Equalization به این صورت است که سعی میکند میزان روشنایی را در کل تصویر پخش کند و نتیجه آن به شکل زیر است: (از دستور `cv.equalizeHist` استفاده می کنیم که از ورودی فقط تصویر یک کانال می گیر پس اول جدا گونه سه کانال را اکولایز می کنیم و سپس دوباره مرج می کنیم)



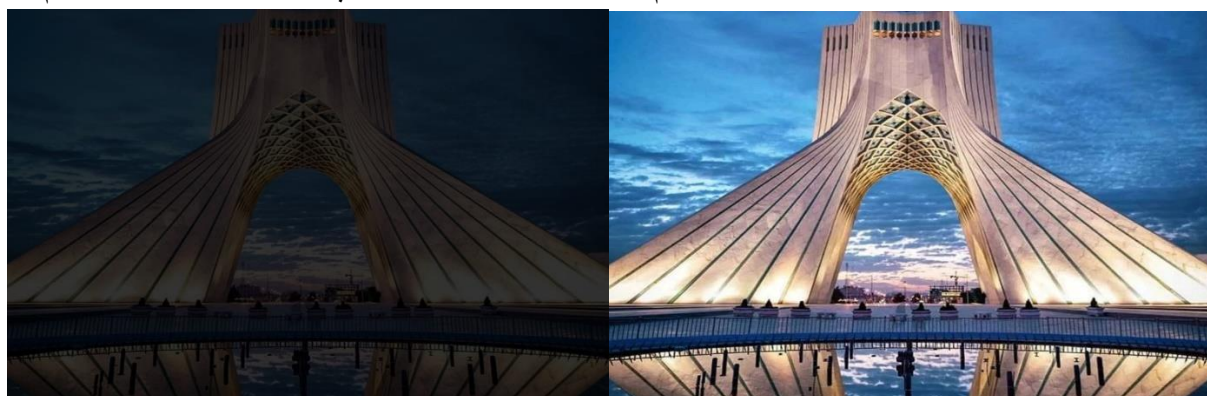
تصاویر اکولایز شده به صورت رنگی و خاکستری

در روش Histogram Stretching مقادیر حداق و حداکثر را پیدا کرده و آن ها را به ۰ و ۲۵۵ مپ می کنیم و همه را به همان نسبت افزایش می دهیم



تصویر قبل و بعد Histogram Stretching

تصویر فرق زیادی نکرده برای اینکه تفاوت بهتر درک کنیم من برای تصویر با کنتراست پایین این کار را تکرار میکنم



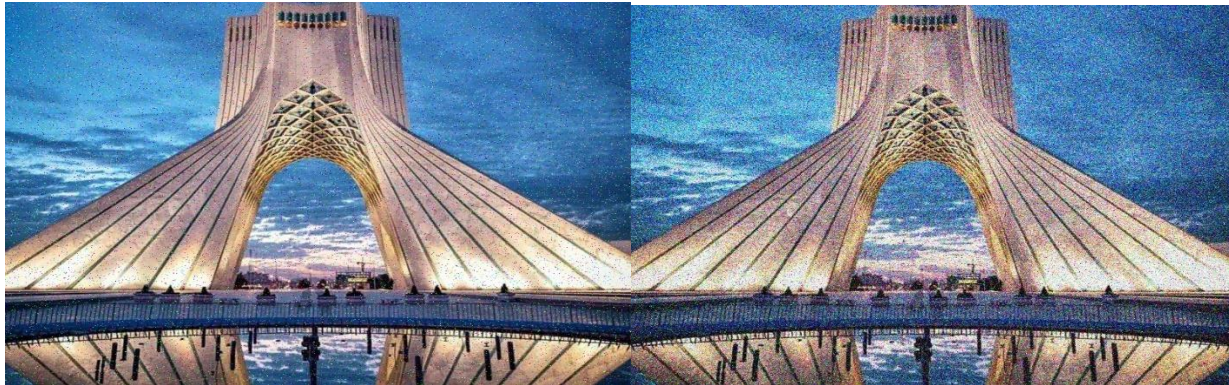
قبل و بعد از HistogramStretching

• PART 2

۱. بارگذاری شد

۲. نویز نمک فلفل در واقع به این شکل است که یک سری پیکسل ها را به صورت رندوم سفید سیاه می کند برای این کار می توان به راحتی آن را در پایتون نوشته و مدل کرد

نویز معروف دیگری که وجود دارد در پردازش تصویر نویز گوسی هست این نویز همانطور که از اسمش مشخص است به شکل توزیع گوسی نویز ایجاد می شود و اگر میانگین صفر باشد این نویز حوالی صفر خواهد بود



سمت راستی نویز گوسی و سمت چپی نویز نمک فلفل

۳. کرنل یک ماتریس معمولاً نسبت به تصویر کوچک است که هدفش استخراج یا فیلتر چیزی در تصویر است و نحوه اعمال آن به این شکل است که یک پیکسل را با پیکسل های همسایه ترکیب می کند و به عدد جدیدی می رسد و آن را جای گذاری میکند

۴. تابع نوشته شد

۵. تابع نوشته شد یکی برای کانولوشنی یکی برای غیر کانولوشنی (مدین فیلتر)

برای اولی به این صورت است که کرنل ما ثابت است و کارش این است که یک عملیات خطی روی پنجره ای از تصویر با

$$\begin{matrix} 6 & 5 & 9 \\ 1 & 4 & 2 \\ 2 & 3 & 4 \end{matrix}$$

توجه به کرنل انجام می دهد مثلاً فرض کنید بخشی از تصویر هستش :

$$\begin{matrix} 1 & 1 & 1 \\ \frac{1}{9} * & 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

حال با اعمال کرنل میانگین گیر که هستش : $\frac{1}{9} \times (6 + 5 + 9 + 1 + 4 + 2 + 2 + 3 + 4)$ که حاصل این عبارت در پیکسل وسطی قرار می گیرد

برای فیلتر مدین همچنین چیزی خواهیم داشت با این تفاوت که عملیات خطی نیست برای همین غیر کانولوشنی نام دارند برای همین مثال بالا و یک کرنل مربعی به این صورت است که

▪ اعداد را کوچک به بزرگ مرتب می کنیم : 1 2 2 3 4 4 5 6 9

▪ میانه (مدین) آن را انتخاب می کنیم : 4

▪ عدد میانه را در پیکسل جای گذاری می کنیم

در آخر سر فیلتر مدین بهتر عمل کرد که می توانید نتایج آن را در صفحه بعد ببینید :



به ترتیب از راست به چپ: فیلتر کانولوشنی, فیلتر مدین, عکس اصلی با نویز نمک فلفل

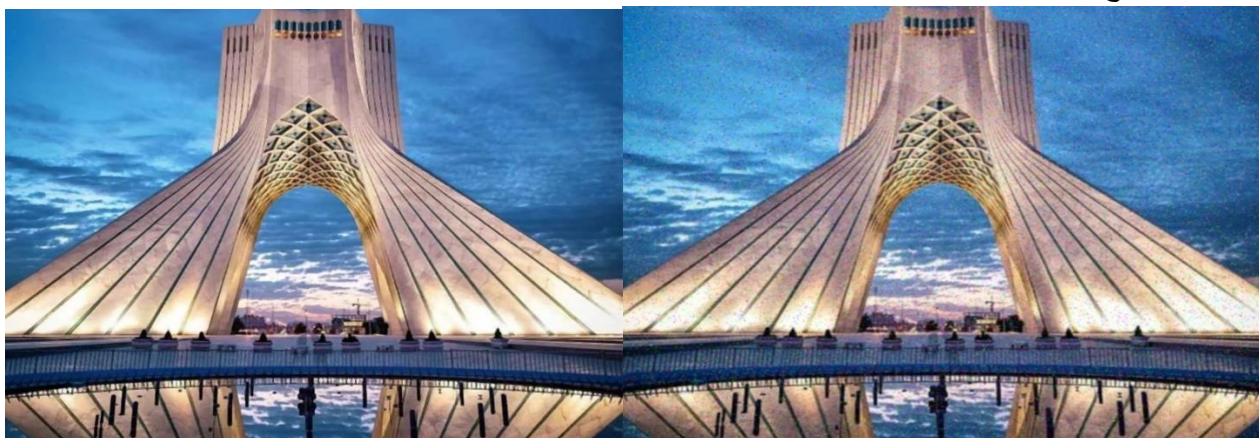
۶. برای فیلتر بالاگذر دو فیلتر لاپلاسین و شارپ انتخاب کردیم که فیلتر شارپ کار لبه گیری را میکند از تصاویر و فیلتر لاپلاسین نوعی مشتق گیری مرتبه دوم است ولی با کرنل ثابت است



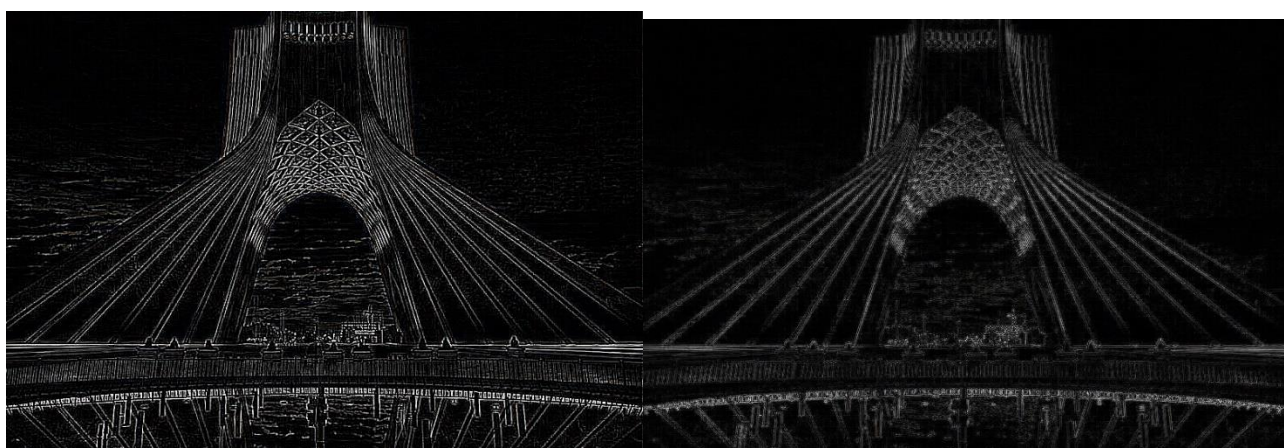
به ترتیب از راست به چپ: فیلتر لاپلاسین, فیلتر شارپ, تصویر اصلی

قابل توجه بود اگر تصویر نویزی می دادیم به فیلتر نویز بیشتر میشد

۷. منطبقا طبق نتایج تصاویر یکی هستند



تصویر از راست به چپ بعد فیلتر اورج گیری و بعد فیلتر مدین



از راست به چپ فیلتر لاپلاسین و فیلتر شارپ

- PART 3

۱. انجام شد

۲. اعداد را مشاهده می کنید:

29.97
309

خروجی خواسته شده

۳. انجام شد

۴. انجام شد